[ ] Start coding or generate with AI.

Linear Regression?

Linear Regression is a supervised machine learning algorithm used to predict a continuous value (number) based on input data.

It finds a straight line that best fits the data.

Linear Regression Equation

$y=mx+b$

Where:

y = Output (dependent variable)

x = Input (independent variable)

m = Slope (how fast y changes with x)

b = Intercept (value of y when x = 0)

Case Study: Predicting Student Marks

Problem Statement

A teacher wants to predict marks of a student based on the number of hours studied.

📊 Dataset

Hours Studied (x)	Marks (y)
1	35
2	40
3	50
4	60
5	65

Observation

As study hours increase, marks also increase

Relationship is approximately linear

How Linear Regression Works (Step-by-Step)

Plot data points on a graph

Draw the best-fit straight line

Minimize the error between actual and predicted values

Use the line equation to predict future values

Example Prediction

Assume the trained model gives:

Predict marks for 4 hours of study

y=7(4)+30

y=7(4)+30=58

Predicted Marks = 58

Python Peactical

```
[ ]
from sklearn.linear_model import LinearRegression

# Data
X = [[1], [2], [3], [4], [5]]
y = [35, 40, 50, 60, 65]

# Model
model = LinearRegression()
model.fit(X, y)

# Prediction
hours = [[4]]
predicted_marks = model.predict(hours)
print(predicted_marks)
```

```
[58.]
```

Advantages of Linear Regression

Easy to understand and implement

Works well for simple relationships

Fast computation

Useful for prediction and trend analysis

Limitations

Limitations

Assumes linear relationship

Sensitive to outliers

Not suitable for complex patterns

Real-Life Applications

House price prediction

Salary prediction

Sales forecasting

Crop yield estimation

Weather trend analysis

Logistic Regression is a supervised machine learning algorithm used for binary classification problems.

It predicts the probability that an input belongs to a particular class (usually 0 or 1) using a sigmoid (S-shaped) function.

Simple Words

Logistic Regression answers Yes/No, True/False, Pass/Fail

Output is not a number like marks, but a probability between 0 and 1

Based on this probability, the final class is decided

Mathematical Form

$$P(y = 1) = \frac{1}{1 + e^{-(wx+b)}}$$

The sigmoid function converts any value into **0–1**

If probability ≥ 0.5 → Class **1**

If probability < 0.5 → Class **0**

[ ]

```
Start coding or generate with AI.
```

Example

Predict Pass (1) or Fail (0) based on study hours

Predict Disease (Yes/No) based on symptoms

Predict Spam (Yes/No) email

Predict Spam (Yes/No) email

How Logistic Regression Works (Simple Explanation)

1. It takes the input feature (**Hours Studied**)
2. Applies a **linear equation**
3. Passes the result through a **Sigmoid function**
4. Produces a probability between **0 and 1**

Sigmoid Function:

$$P(y=1) = \frac{1}{1+e^{-z}}$$

Where:

$$z = w \cdot x + b$$

🔍 Decision Rule

- If probability ≥ 0.5 → **Pass**
- If probability < 0.5 → **Fail**

🖊️ Example Prediction

Student studies for 3 hours

- Model output = **0.30**
- Prediction → ❌ Fail

Student studies for 5 hours

- Model output = **0.85**
- Prediction → ✅ Pass

Python Practical

```
[ ]
```

```python
from sklearn.linear_model import LogisticRegression

# Training data
X = [[1], [2], [3], [4], [5], [6]]
y = [0, 0, 0, 1, 1, 1]

# Model
model = LogisticRegression()
model.fit(X, y)

# Prediction
```

```
hours = [[5]]
prediction = model.predict(hours)
print("Pass" if prediction[0] == 1 else "Fail")
```

```
Pass
```

Advantages of Logistic Regression

Easy to understand

Fast to train

Works well for binary problems

Probability-based output

Limitations

Works best when classes are linearly separable

Not suitable for complex non-linear data

A hospital wants to predict whether a patient has a disease or not based on basic medical test results.

This is a binary classification problem:

Disease = 1

No Disease = 0

Dataset (Simple Example)

Age	Blood Sugar Level	Disease
25	120	0
30	130	0
40	150	1
50	170	1
60	180	1

Logistic Regression Works (Simple Steps)

Takes patient data (Age, Sugar Level)

Applies a linear equation

Uses a sigmoid function

Uses sigmoid function

Outputs probability between 0 and 1

Classifies patient as Disease / No Disease

Q.1 predict whether a patient has a disease or not using multiple medical features with the help of Logistic Regression.

The model uses **four medical features**:

Feature	Description
Age	Patient age (years)
Blood Sugar	Sugar level (mg/dL)
Blood Pressure	BP (mmHg)
BMI	Body Mass Index

Logistic Regression?

Output has two classes only

Can handle multiple input features

Produces probability-based decision

Widely used in healthcare diagnosis

Difference Between Linear Regression and Logistic Regression

Linear Regression	Logistic Regression
Predicts a **continuous value**	Predicts a **binary outcome**
Output is a **number**	Output is **0 or 1 (Yes/No)**

Output Type

Aspect	Linear	Logistic
Output Range	$-\infty$ to $+\infty$	0 to 1
Example	Marks = 78	Pass (1) / Fail (0)

Mathematical Model

Linear Regression

$$y = mx + b$$

Logistic Regression

$$P(y = 1) = \frac{1}{1 + e^{-(mx+b)}}$$

(Logistic regression uses a **Sigmoid function**)

- Problem Type

Linear Regression	Logistic Regression
House price prediction	Email spam detection
Salary prediction	Disease (Yes/No)
Temperature forecasting	Pass / Fail result

Graph Shape

Linear Regression ⟶ Straight line

Logistic Regression ⟶ S-shaped (Sigmoid) curve

- Learning Method

Feature	Linear	Logistic
Error Function	Mean Squared Error (MSE)	Log Loss (Cross-Entropy)
Decision Boundary	Not required	Required

Linear Regression predicts continuous values, whereas Logistic Regression predicts categorical (binary) outcomes using a sigmoid function.

Use Linear Regression ⟶ when output is a number

Use Logistic Regression ⟶ when output is Yes/No

- Design a machine learning model using Logistic Regression to predict whether a crop is at risk of disease (Yes/No) based on environmental and crop-related features.

Feature	Description
Temperature (°C)	Average weekly temperature
Humidity (%)	Relative humidity
Rainfall (mm)	Weekly rainfall
Wind Speed (km/h)	Average wind speed
Crop Age (days)	Days after sowing

Output (Target Variable)

Value	Meaning
1	Disease Risk Present
0	No Disease Risk

Temp	Humidity	Rainfall	Wind	Crop Age	Disease
28	65	20	5	18	0
30	70	25	6	22	0
32	85	80	4	30	1
33	90	95	3	35	1
31	88	70	4	28	1

Collect weather and crop data

Apply a linear combination of features

Use sigmoid function

Obtain disease risk probability

Classify crop as Risk / No Risk

Early Prediction of Crop Disease Risk Using Machine Learning with Multi-Feature Environmental Data

Early Prediction of Water Quality and Contamination Risk Using Machine Learning

Background of the Problem

Access to clean and safe water is a major global challenge, particularly in developing countries. Rivers, lakes, groundwater, and drinking water sources are increasingly contaminated due to:

Industrial discharge

Agricultural runoff (fertilizers and pesticides)

Urban sewage and wastewater

Poor monitoring infrastructure

Traditional water quality assessment relies on laboratory-based chemical analysis, which is:

Time-consuming

Costly

Not suitable for real-time monitoring

Difficult to scale across large regions

As a result, water contamination is often detected too late, leading to water-borne diseases, environmental damage, and public health crises.

Water Quality Checking Device Names (Real Devices)

Water quality can be monitored using devices such as a multiparameter water quality meter, water quality analyzer, portable testing kit, and IoT-based water quality monitoring systems.

Logistic Regression for Water-Borne Diseases?

Why Logistic Regression Fits Water-Borne Disease Problems

Water-borne disease prediction is usually a binary decision problem:

Disease Outbreak = Yes / No

Patient Infected = Yes / No

Water Safe = Safe / Unsafe

Logistic Regression Is Used (Conceptually)

Logistic Regression predicts the probability of disease occurrence based on multiple environmental, water quality, and demographic factors.

If probability ≥ threshold (e.g., 0.5) ⟶ High risk / Disease present

Input Features (Multiple, Real)

Water Quality Parameters

pH

Turbidity

Total Dissolved Solids (TDS)

Dissolved Oxygen (DO)

Nitrate level

Fecal coliform count

Environmental Factors

Rainfall

Temperature

Flood events

Human & Demographic Factors

Population density

Sanitation level

Water source type

Output Variable

Value	Meaning
1	Water-borne disease outbreak / infection
0	No outbreak / healthy population

Logistic Regression Model (Conceptual)

$$P(\text{Disease}) = \frac{1}{1 + e^{-(w_1 pH + w_2 Turb + w_3 TDS + w_4 Rain + w_5 Pop + b)}}$$

Real Water-Borne Diseases Where Logistic Regression Is Used

1. Cholera

Common Symptoms (Features)

Profuse watery diarrhea (rice-water stool)

Severe dehydration

Vomiting

Muscle cramps

Dry mouth and thirst

Low blood pressure

2. Typhoid

Common Symptoms (Features)

High fever (often >39°C)

Headache

Weakness and fatigue

Abdominal pain

Loss of appetite

Constipation or diarrhea

3. Dysentery (डिसेन्ट्री)

Common Symptoms (Features)

Bloody diarrhea

Mucus in stool

Abdominal cramps

Fever

Nausea

Dehydration

3. Hepatitis A

Common Symptoms (Features)

Jaundice (yellow eyes/skin)

Dark urine

Pale stool

Fever

Fatigue

Loss of appetite

Nausea

4. Giardiasis(जियार्डायसिस)

Common Symptoms (Features)

Chronic diarrhea

Greasy/foul-smelling stool

Abdominal bloating

Gas

Weight loss

Fatigue

📊 Combined Feature Set (For ML / Logistic Regression)

Feature	Type
Fever	Binary / Continuous
Diarrhea	Binary / Frequency
Vomiting	Binary
Dehydration	Severity
Abdominal Pain	Severity
Jaundice	Binary
Stool Type	Categorical
Fatigue	Binary
Appetite Loss	Binary

Target Variable (Output)

Value Meaning

1 Water-borne disease present

0 No water-borne disease

In Logistic Regression for water-borne diseases:

Symptoms = independent variables (features)

Disease presence = dependent variable (Yes/No)

Step 1: Install Required Libraries

[ ]

```
pip install numpy pandas scikit-learn shap matplotlib
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.50.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
```

```
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.1)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap) (0.43.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

Step 2: Import Libraries

```
import numpy as np
import pandas as pd
import shap
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

Step 3: Create Sample Water Quality Dataset

```
data = {
    "pH": [6.8, 7.1, 6.2, 5.9, 7.4, 6.0, 6.5, 5.8],
    "Turbidity": [3, 2, 8, 10, 1, 9, 6, 11],
    "TDS": [300, 280, 700, 820, 250, 760, 680, 900],
    "DO": [6.5, 7.0, 3.8, 3.2, 7.2, 3.5, 4.0, 2.8],
    "Nitrate": [20, 18, 45, 60, 15, 55, 48, 70],
    "Disease_Risk": [0, 0, 1, 1, 0, 1, 1, 1]
}

df = pd.DataFrame(data)
```

Step 4: Split Features and Target

```
X = df.drop("Disease_Risk", axis=1)
y = df["Disease_Risk"]
```

Step 5: Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

Step 6: Feature Scaling

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Step 7: Train Logistic Regression Model

```
model = LogisticRegression()
```

```
model.fit(X_train_scaled, y_train)
```

```
LogisticRegression
LogisticRegression()
```

Step 8: Make Prediction (Example)

```
sample = pd.DataFrame({
    "pH": [6.1],
    "Turbidity": [9],
    "TDS": [780],
    "DO": [3.2],
    "Nitrate": [58]
})

sample_scaled = scaler.transform(sample)
prediction = model.predict(sample_scaled)
probability = model.predict_proba(sample_scaled)

print("Prediction:", "High Risk" if prediction[0] == 1 else "Low Risk")
print("Probability:", probability)
```

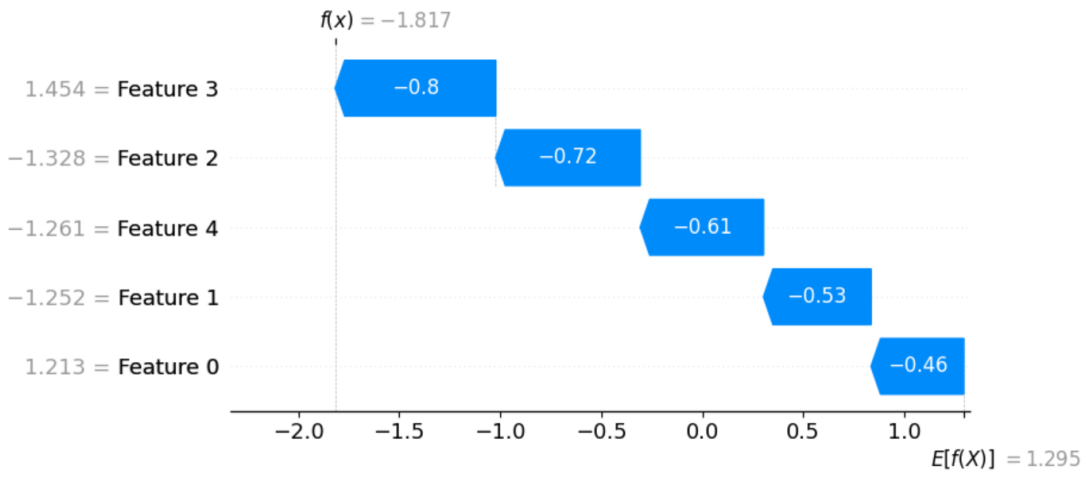
```
Prediction: High Risk
Probability: [[0.04670799 0.95329201]]
```

Step 9: Apply XAI Using SHAP

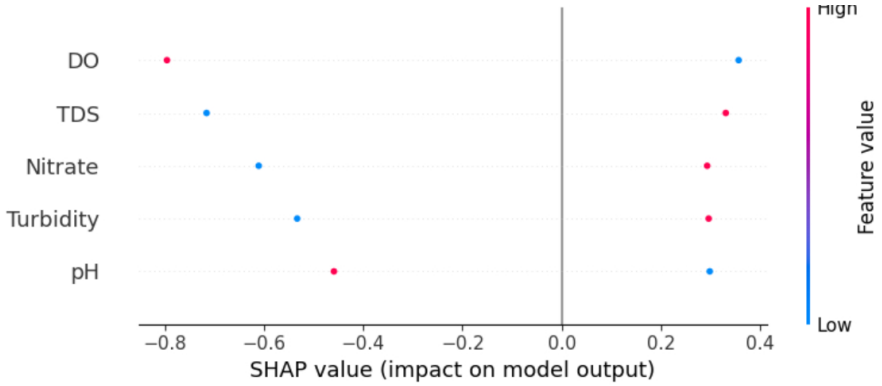
```
explainer = shap.Explainer(model, X_train_scaled)
shap_values = explainer(X_test_scaled)
```

Step 10: SHAP Feature Importance (Global Explanation)

```
# shap.plots.waterfall(shap_values[0], feature_names=X.columns)
shap.plots.waterfall(shap_values[0])
```




```
shap.summary_plot(shap_values, X_test, feature_names=X.columns)
```

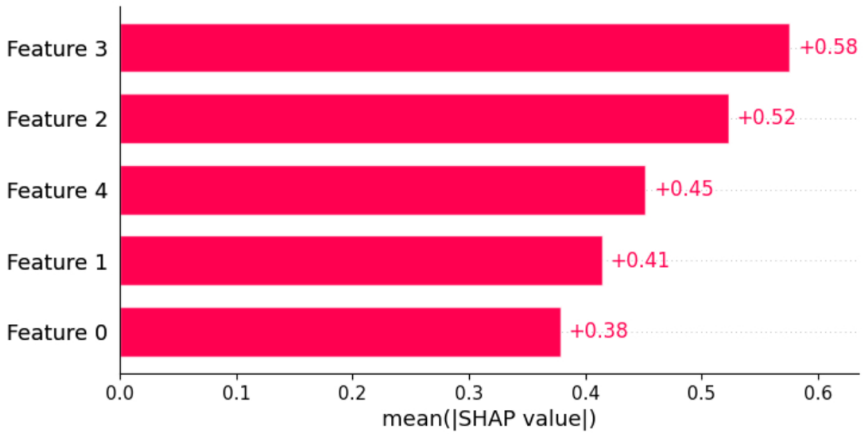


SHAP Bar Plot (Mean Importance)

Average contribution of each feature

Clean and simple (journal-friendly)

```
shap.plots.bar(shap_values)
```



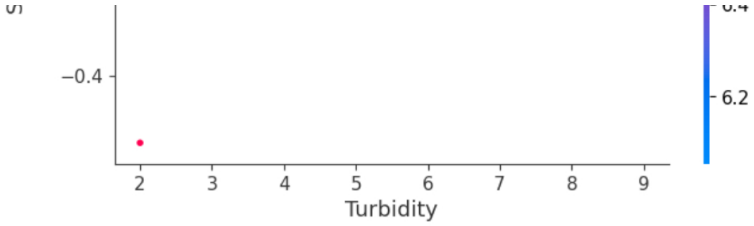
3 SHAP Dependence Plot (Feature Interaction)

What it shows

How one feature affects prediction

Relationship between feature value and risk

```
shap.dependence_plot(
    "Turbidity", shap_values.values, X_test
)
```



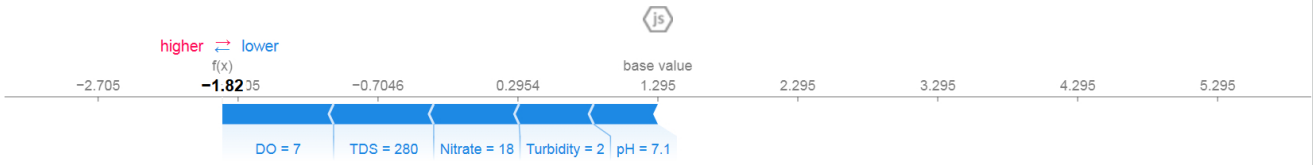
4. SHAP Force Plot (Single Prediction – Interactive)

What it shows

Push & pull of features for one sample

Best for presentations and demos

```
shap.initjs()
shap.force_plot(
    shap_values.base_values[0],
    shap_values.values[0],
    X_test.iloc[0]
)
```




```
Start coding or generate with AI.
```

What is SVM?

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression.

It works by finding an optimal decision boundary (hyperplane) that maximizes the margin between different classes.

Intuition (Simple Words)

SVM draws a line (2D) or plane (3D) or hyperplane (high-D)

The best boundary is the one with the maximum distance from the nearest points of both classes

These nearest points are called support vectors

Mathematical Idea (Classification)

For binary classification, SVM solves:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to } y_i(w \cdot x_i + b) \geq 1$$

- w: weight vector (orientation of hyperplane)
- b: bias
- x_i: data point
- $y_i \in \{-1, +1\}$: class label

👉 This **maximizes the margin**.

What are Support Vectors?

Data points closest to the decision boundary

They define the position of the hyperplane

Removing other points does not change the boundary

Kernels (Key Strength of SVM)

When data is not linearly separable, SVM uses kernels to map data to a higher dimension.

Kernel	Use Case
Linear	Linearly separable data
Polynomial	Curved boundaries
RBF (Gaussian)	Complex patterns (most popular)
Sigmoid	Neural-network-like behavior

Simple Example

Problem: Classify emails as Spam / Not Spam

Features: word frequency, links, sender score

SVM: finds a boundary that best separates spam vs non-spam with maximum margin.

[ ]

```
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = SVC(kernel="rbf", C=1.0, gamma="scale")
model.fit(X_train, y_train)
```

```
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9473684210526315
```

How to Change Kernel in Support Vector Machine (SVM)

In scikit-learn, you change the kernel by modifying the kernel parameter in SVC().

1 Linear Kernel (for linearly separable data)

```
from sklearn.svm import SVC

model = SVC(kernel="linear", C=1.0)
model.fit(X_train, y_train)
```

```
SVC(kernel='linear')
```

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.956140350877193
```

One-Line Shortcut (Also Correct)

```
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.956140350877193
```

Accuracy

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Example:

- Correct predictions = 90
- Total samples = 100

$$Accuracy = 90/100 = 0.90 = 90\%$$

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Train model
```

```
model = SVC(kernel="linear", C=1.0)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

```
Accuracy: 0.96
```

Naive Bayes

Naive Bayes is a supervised machine learning algorithm based on Bayes' Theorem.

It is mainly used for classification problems, especially text and categorical data.

It is called "Naive" because it assumes that all features are independent of each other.

Simple Intuition

Uses probability to predict a class

Calculates the probability of each class

Chooses the class with the highest probability

Bayes' Theorem

$$P(C|X) = \frac{P(X|C)\,P(C)}{P(X)}$$

Where:

- $P(C|X)$: Posterior probability (class after seeing data)
- $P(X|C)$: Likelihood
- $P(C)$: Prior probability
- $P(X)$: Evidence

Types of Naive Bayes

Type	Used When
Gaussian NB	Continuous data (marks, height)
Multinomial NB	Text data (word counts)
Bernoulli NB	Binary features (Yes/No)

Problem

Predict whether a student **Pass / Fail** based on **Study Hours**.

Hours Studied	Result
Low	Fail
Medium	Pass
High	Pass

Naive Bayes computes:

- Probability of **Pass**
- Probability of **Fail**
- Chooses the class with **higher probability**

[ ]

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import pandas as pd

# Sample data
data = {
    "Hours": [1, 2, 3, 4, 5, 6],
    "Result": [0, 0, 0, 1, 1, 1]
}

df = pd.DataFrame(data)

X = df[["Hours"]]
y = df["Result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.0
```

Advantages of Naive Bayes

Very fast

Works well with text data

Requires less training data

Easy to implement

Where Naive Bayes is Used

Spam email detection

Sentiment analysis

Document classification

Medical diagnosis (baseline model)

Aspect	Naive Bayes	SVM
Speed	Very fast	Slower
Data size	Small–medium	Medium–large
Interpretability	Medium	Low
Text data	Excellent	Moderate

Problem Statement

A college wants to predict whether a student will Pass or Fail based on simple academic features using the Naive Bayes algorithm.

This is a binary classification problem:

Pass = 1

Fail = 0

Dataset (Simple & Realistic)

Study Hours	Attendance (%)	Internal Marks	Result
Low	60	40	Fail
Medium	70	55	Fail
Medium	80	65	Pass
High	85	75	Pass
High	90	80	Pass

Features Used (Input Data)

Feature	Description
Study Hours	Hours studied per day
Attendance	Percentage of attendance
Internal Marks	Marks in internal exams

Why Naive Bayes?

Works well with small datasets

Easy to understand

Fast computation

Suitable for categorical data

Good baseline classifier

How Naive Bayes Works (Simple Steps)

Calculate probability of Pass and Fail

Calculate probability of each feature for each class

Apply Bayes' Theorem

Choose the class with highest probability

Bayes' Theorem Used

$$P(Class|Features) = \frac{P(Features|Class) \times P(Class)}{P(Features)}$$

Naive assumption:

> All features are **independent** of each other

Example Prediction

New Student Details:

- Study Hours = **Medium**
- Attendance = **75%**
- Internal Marks = **60**

Naive Bayes Output:

- $P(\text{Pass}) = 0.78$
- $P(\text{Fail}) = 0.22$

✅ **Prediction: PASS**

Double-click (or enter) to edit

[ ]

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```python
# Dataset
data = {
    "StudyHours": ["Low", "Medium", "Medium", "High", "High"],
    "Attendance": [60, 70, 80, 85, 90],
    "InternalMarks": [40, 55, 65, 75, 80],
    "Result": ["Fail", "Fail", "Pass", "Pass", "Pass"]
}

df = pd.DataFrame(data)

# Encode categorical data
le = LabelEncoder()
df["StudyHours"] = le.fit_transform(df["StudyHours"])
df["Result"] = le.fit_transform(df["Result"])

X = df[["StudyHours", "Attendance", "InternalMarks"]]
y = df["Result"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Naive Bayes model
model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.5
```

Metrics Implementation for Naive Bayes (NB)

```python
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    fbeta_score,
    classification_report
)
```

Step 2: Predict Using the Trained NB Model

(Using the model you already trained)

Double-click (or enter) to edit

```
y_pred = model.predict(X_test)
```

Step 3: Calculate Evaluation Metrics

Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.5
```

Precision

```
precision = precision_score(y_test, y_pred)
print("Precision:", precision)
```

```
Precision: 0.5
```

Recall

```
recall = recall_score(y_test, y_pred)
print("Recall:", recall)
```

```
Recall: 1.0
```

F1-score

```
f1 = f1_score(y_test, y_pred)
print("F1-score:", f1)
```

```
F1-score: 0.6666666666666666
```

All Metrics Together (Recommended)

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.00      0.00      0.00         1
           1       0.50      1.00      0.67         1

    accuracy                           0.50         2
   macro avg       0.25      0.50      0.33         2
weighted avg       0.25      0.50      0.33         2

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Metric	Meaning
Accuracy	Overall correctness of the model

Precision	How many predicted Pass are actually Pass
Recall	How many actual Pass students are detected
F1-score	Balance between Precision & Recall
F2-score	Recall-weighted score (missed Pass is costly)

Accuracy ⟶ overall correctness

Precision ⟶ correctness of positive predictions

Recall ⟶ ability to find all positives

F1 ⟶ balance of precision & recall

F2 ⟶ recall-focused metric

Decision Tree

Decision Tree is a supervised machine learning algorithm used for classification and regression.

It works like a flowchart, where:

Internal nodes ⟶ questions (conditions)

Branches ⟶ answers (Yes/No)

Leaf nodes ⟶ final decision/output

Simple Intuition

Think of how we make decisions in real life:

If marks ≥ 40 ⟶ Pass

Else ⟶ Fail

A Decision Tree learns such if–else rules automatically from data.

Types of Decision Trees

Classification Tree ⟶ output is a class (Pass/Fail, Yes/No)

Regression Tree ⟶ output is a number (marks, price)

How Decision Tree Works

Splits data based on the best feature

Uses measures like:

Gini Index

Entropy

Information Gain

Continues splitting until a stopping condition is met

Problem

Predict whether a student will Pass or Fail based on:

Study Hours

Attendance

Study Hours	Attendance (%)	Result
Low	60	Fail
Medium	70	Fail
Medium	80	Pass
High	85	Pass
High	90	Pass

Decision Rule (Learned by Tree)

If Attendance ≥ 75:

Pass

Else:

Fail

Simple Python Code

[ ]

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score

# Dataset
data = {
    "StudyHours": ["Low", "Medium", "Medium", "High", "High"],
    "Attendance": [60, 70, 80, 85, 90],
    "Result": ["Fail", "Fail", "Pass", "Pass", "Pass"]
}

df = pd.DataFrame(data)

# Encode categorical feature
le = LabelEncoder()
df["StudyHours"] = le.fit_transform(df["StudyHours"])
df["Result"] = le.fit_transform(df["Result"])
```

```
X = df[["StudyHours", "Attendance"]]
y = df["Result"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Decision Tree model
model = DecisionTreeClassifier(criterion="gini")
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 1.0
```

Advantages of Decision Tree

Easy to understand and interpret

No need for feature scaling

Works with categorical and numerical data

Visualizable

Limitations

Can overfit the data

Small change in data → different tree

Less stable than ensemble method

Algorithm	Interpretability	Overfitting
Naive Bayes	Medium	Low
Decision Tree	High	High
Random Forest	Medium	Low

Double-click (or enter) to edit